

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS
Facultad de Ciencias Puras · Carrera de Ingeniería Informática
Potosí — Bolivia

INF 220 — PROGRAMACIÓN ORIENTADA A OBJETOS Y ESTRUCTURAS DE DATOS

GUÍA DE LABORATORIO N.º 4

Métodos: parámetros, retorno, sobrecarga y métodos estáticos

Unidad	Unidad 1 — Programación Orientada a Objetos
Tema	Paso de parámetros · retorno de valores · sobrecarga de métodos · métodos estáticos vs. de instancia
Requisito	Guía de Laboratorio N.º 3 (constructores y encapsulamiento)
Stack	JDK Temurin 25 (LTS) · Visual Studio Code · Extension Pack for Java
Duración	1 sesión de laboratorio
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán

1 Objetivos

► Objetivo general

Diseñar e implementar métodos en Java comprendiendo el paso de parámetros, el retorno de valores, la sobrecarga y la diferencia entre métodos estáticos y de instancia, para descomponer problemas en piezas reutilizables.

► Objetivos específicos

- Definir métodos con y sin parámetros, con y sin valor de retorno.
- Comprender que en Java el paso de parámetros es siempre por valor.
- Sobrecargar métodos: mismo nombre con distinta lista de parámetros.
- Distinguir un método estático de uno de instancia y saber cuándo usar cada uno.
- Descomponer un programa en métodos claros y reutilizables.

2 Requisitos previos

Software	JDK Temurin 25 + Visual Studio Code con Extension Pack for Java.
Conocimientos	Clases, objetos, atributos, encapsulamiento y constructores (Guías 2 y 3).

3 Marco teórico

► Anatomía de un método

Un método agrupa un bloque de instrucciones bajo un nombre para poder invocarlo cuando se necesite. Su cabecera define cómo se lo usa:

```
public  int  sumar(int a, int b)  { ... }
```

modific. tipo nombre parámetros
 de
 retorno

DEFINICIÓN · Firma del método

La **firma** es el nombre del método más el tipo y orden de sus parámetros (por ejemplo `sumar(int, int)`). Es lo que permite a Java saber qué método invocar. El tipo de retorno *no* forma parte de la firma.

► Parámetros y argumentos

- **Parámetro** — la variable declarada en la cabecera del método (`int a`).
- **Argumento** — el valor concreto que se envía al llamarlo (`sumar(3, 5)`).

► Retorno de valores

Un método puede devolver un resultado con `return`, declarando el tipo correspondiente; o no devolver nada, en cuyo caso su tipo de retorno es `void`.

► Paso de parámetros por valor

Java **siempre** pasa los argumentos **por valor**: el método recibe una copia. Con tipos primitivos, modificar el parámetro no afecta a la variable original. Con objetos, lo que se copia es el valor de la *referencia*, así que el método puede modificar el objeto apuntado (aunque no puede hacer que la variable externa apunte a otro objeto).

► Sobrecarga de métodos

Una clase puede tener varios métodos con el mismo nombre siempre que difieran en el número o el tipo de sus parámetros. Java elige el correcto según los argumentos que reciba.

IMPORTANTE · El tipo de retorno no basta

Dos métodos que solo se diferencien en el tipo de retorno **no** son una sobrecarga válida: `int f(int)` y `double f(int)` provocan un error de compilación, porque tienen la misma firma.

► Métodos estáticos y de instancia

De instancia	Pertenecen al objeto: necesitan un objeto creado con <code>new</code> para invocarse (<code>obj.metodo()</code>). Pueden usar los atributos del objeto.
Estáticos	Llevan <code>static</code> y pertenecen a la clase: se invocan con <code>Clase.metodo()</code> , sin crear objeto. No pueden usar atributos de instancia. Ideales para utilidades (<code>Math.max</code>).

4 Desarrollo de la práctica

Trabaja en una carpeta `Lab04`.

► Actividad 1 — Métodos con y sin retorno

Crea una clase Utilidades con un método que solo actúa (void) y otro que devuelve un valor:

```
public class Utilidades {
    void saludar(String nombre) {           // sin retorno
        System.out.println("Hola, " + nombre);
    }

    int cuadrado(int n) {                   // con retorno
        return n * n;
    }
}
```

Úsalos desde Principal.java:

```
Utilidades u = new Utilidades();
u.saludar("Ana");
int r = u.cuadrado(5);
System.out.println("El cuadrado es: " + r);
```

► Actividad 2 — Paso de parámetros por valor

Comprueba el comportamiento del paso por valor con un primitivo y con un objeto:

```
public class Paso {
    static void modificar(int x) {
        x = 100;           // solo cambia la copia local
    }

    public static void main(String[] args) {
        int numero = 5;
        modificar(numero);
        System.out.println(numero); // sigue siendo 5
    }
}
```

EJEMPLO • Con objetos es distinto

Si el método recibe un objeto (por ejemplo un Estudiante) y llama a `e.setEdad(...)`, el cambio **sí** se refleja fuera, porque ambos manejan la misma referencia al objeto.

► Actividad 3 — Sobrecarga de métodos

Agrega a Utilidades tres versiones del método sumar:

```
int sumar(int a, int b) {
    return a + b;
}
double sumar(double a, double b) {
    return a + b;
}
int sumar(int a, int b, int c) {
    return a + b + c;
}
```

Invoca las tres desde el main y observa cómo Java elige la versión adecuada según los argumentos:

```
System.out.println(u.sumar(2, 3));      // usa la de dos int
System.out.println(u.sumar(2.5, 3.5));  // usa la de dos double
System.out.println(u.sumar(1, 2, 3));    // usa la de tres int
```

► Actividad 4 — Métodos estáticos

Creas una clase utilitaria con métodos estáticos, que se usan sin instanciar ningún objeto:

```
public class MathUtil {
    public static int max(int a, int b) {
        return (a > b) ? a : b;
    }

    public static double promedio(double[] valores) {
        double suma = 0;
        for (double v : valores) {
            suma += v;
        }
        return suma / valores.length;
    }
}
```

Se invocan a través del nombre de la clase, no de un objeto:

```
System.out.println(MathUtil.max(7, 3));
double[] notas = {60, 75, 90};
System.out.println(MathUtil.promedio(notas));
```

► Actividad 5 — Descomponer un programa en métodos

Escribe un programa que lea 5 notas y use métodos separados para calcular el promedio y decidir si el estudiante aprobó, manteniendo el main breve y legible:

```
import java.util.Scanner;

public class Evaluacion {
    static double promedio(double[] n) {
        double s = 0;
        for (double x : n) s += x;
        return s / n.length;
    }
    static boolean aprobo(double prom) {
        return prom >= 51;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double[] notas = new double[5];
        for (int i = 0; i < 5; i++) {
            System.out.print("Nota " + (i + 1) + ": ");
            notas[i] = sc.nextDouble();
        }
        double p = promedio(notas);
        System.out.println("Promedio: " + p);
        System.out.println(aprobo(p) ? "Aprobó" : "Reprobó");
        sc.close();
    }
}
```

5 Bibliografía y recursos

- Deitel, P. & Deitel, H. — Cómo programar en Java. Pearson.
- Sierra, K. & Bates, B. — Head First Java. O'Reilly.
- Métodos (tutorial oficial): <https://docs.oracle.com/javase/tutorial/java/javaOO/methods.html>